# University of Cape Town
# Department of Computer Science

Computer Science CSC3003S
Class Test 1, 2017

**Answer all questions.**
**Approximate marks per question are shown in brackets.**

**Time: 40 minutes** **Marks: 40**

## Section 1. Project Management [22 Marks]

## Question 1 [10 Marks]

a. How do you set out the manage risk at the start of the project? [4]

**Answer**:

1. Identify specific risks to the project [1]
2. Analyze the risks [1]
3. Rank them in a particular order [1]
4. Plan for monitoring, mitigation, management. [1]

b. List the 3M's of risk management and explain what they entail. [6]

Answer:

Mitigation [1]: avoiding or reducing risk [0.5] by changing requirements or transferring it (e.g. buy insurance), or assume the risk and put in measures to control it [0.5]

Monitoring [1]: what factors can we track that will enable us to determine if the risk is becoming more or less likely [1]

Management [1]: What contingency plans do we have that will enable us to determine if risk is becoming more or less likely? [1]

## Question 2 [12 Marks]

a. Consider the following principles of Extreme Programming (XP): [5]

Incremental Planning, Small Releases, Simple Design, Test-first Development, Refactoring, Pair Programming, Collective Ownership, Continuous Integration, Sustainable Pace, On-site customer

Briefly describe each principle. (0.5 each)

**Answer**:

Incremental Planning: Requirements are recorded on story cards and the stories to be included in a release are determined by the time available and their relative priority. The developers break these stories into development 'Tasks'.

Small Releases: The minimal useful set of functionality that provides business value is developed first. Releases of the system are frequent and incrementally add functionality to the first release.

Simple Design: Enough design is carried out to meet the current requirements and no more.

Test-first Development: An automated unit test framework is used to write tests for a new piece of functionality before that functionality itself is implemented.

Refactoring: All developers are expected to refactor the code continuously as soon as possible code improvements are found. This keeps the code simple and maintainable.

Pair Programming: Developers work in pairs, checking each other's work and providing the support to always do a good job.

Collective Ownership: The pairs of developers work on all areas of the system, so no islands of expertise develop and all the developers take responsibility for all of the code: anyone can change anything.

Continuous Integration: As soon as the work on a task is complete, it is integrated into the whole system. After any such integration, all the unit tests in the system must pass.

Sustainable Pace: Large amounts of overtime are not acceptable as the net effect is often to reduce code quality & medium term productivity

On-site Customer: A representative of the end-user of the system (the customer) should be available full time for the use of the XP team. In an extreme programming process, the customer is a member of the development team and is responsible for bringing system requirements to the team for implementation.

b. Describe test-driven development. [4]

**Answer**:

**Points Include**: (1 mark each, max 4).

Productivity is similar to that of two people working independently

Common ownership of code

Collective (team) responsibility for the system ( resolving problems).

Spreads knowledge across the team.

Reduces risk if someone leaves

Motivates refactoring as the whole team will benefit from it.

c. List some of the benefits of pair programming as found in XP [3]

**Answer**:

**Points Include**: (1 mark each, max 3).

Productivity is similar to that of two people working independently

Common ownership of code

Collective (team) responsibility for the system (resolving problems).

Spreads knowledge across the team.

Reduces risk if someone leaves

Motivates refactoring as the whole team will benefit from it.

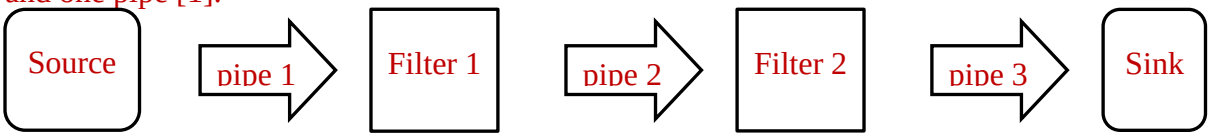
## Section 2. Architecture [18 Marks]

## Question 3 [12 Marks]

a. Briefly describe pipe and filter architecture and draw a diagram to illustrate it. [6]

**Answer**: A system is decomposed into a set of functional transformations that consume inputs and produce outputs. Data flows from one function to another (the pipeline) [1] and is transformed as it passes through the sequence [1].

+ [2] for diagram shape (source and sink are not required) and for label at least one filter [1] and one pipe [1].




b. How can the pipe and filter architecture be generalized, what is it then called? [2]

**Answer**: It can be generalized to have branching pipes connecting filters, (Or they can draw a diagram) [1]. It is then called a Data Flow Architecture. [1]

c. We discussed several architecture patterns. What is the key aim of these patterns? Define any new terms that you introduce. [4]

**Answer**: The key aim of architectural patterns is to minimize coupling [1] while maximizing cohesion [1]. Cohesion is the degree to which communication takes place within the module [1]. Coupling is the degree to which communication takes place between modules [1]. Can also explain in terms of "Separation of concerns and localization of impact", but then they must say what they mean by this.

## Question 4 [6 Marks]

Many personal computer applications (like word processors or games) are event processing systems.

a. Why are they called "event processing systems"? [1]

**Answer**: They respond to events in the environment of the system (also ok to say respond to user events) [1].

b. What is the key characteristic of "event processing systems" that impacts the architecture? [2]

**Answer**: Their key characteristic is that event timing [1] is unpredictable [1] so the architecture has to be organised to handle this. [1]. (max 2 marks)

c. Please describe a high-level view of data processing systems. [3]

**Answer**: Data processing systems are data-driven [1] and operate in batch mode [1] and generally have an input-process-output structure [2]. Records are input into the system [1], the information is processed [1] and outputs are generated [1]. (Max 3)